LAB # 9: To design the traffic light controller using VHDL and reproduce the output onthe FPGA board

Objectives

The objective of this lab is to *design* and test traffic light controller using VHDL.

Part 1 - Introduction, specification, design and testing of two way traffic light controller

Digital controllers are good examples of circuits that can be efficiently implemented when modelled as state machines. In the present experiment, we want to design a Traffic Light Controller (TLC) with the following characteristics and also summarized in the table 9.1

Three modes of operation: Regular, Test, and Standby.

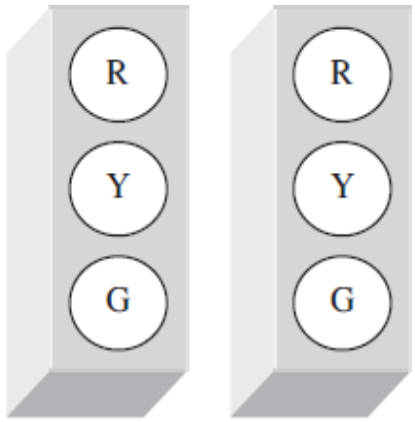
Regular Mode: Four states, each with an independent, programmable time, passed to the circuit by means of a CONSTANT.

Test mode: allows all pre-programmed times to be overwritten (by a manual switch) with a small value, such that the system can be easily tested during maintenance (1 second per state). This value should also be programmable and passed to the circuit using a CONSTANT.

Standby Mode: If set (by a sensor accusing malfunctioning, for example, or a manual switch) the system should activate the yellow lights in both directions and remain so while the standby signal is active.

Assume that a 60 Hz clock is available. However, you can also use clock divider to acquire the desired clock frequency.

Table 9.1 Specification of TLC



State	Operation Mode		
	Regular	Test	Standby
	Time	Time	Time
RG	timeRG(30 s)	timeTEST(1 s)	--
RY	timeRY(5 s)	timeTEST(1 s)	--
GR	timeGR(45 s)	timeTEST(1 s)	--
YR	timeYR(5 s)	timeTEST(1 s)	--
YY	--	--	Indefinte

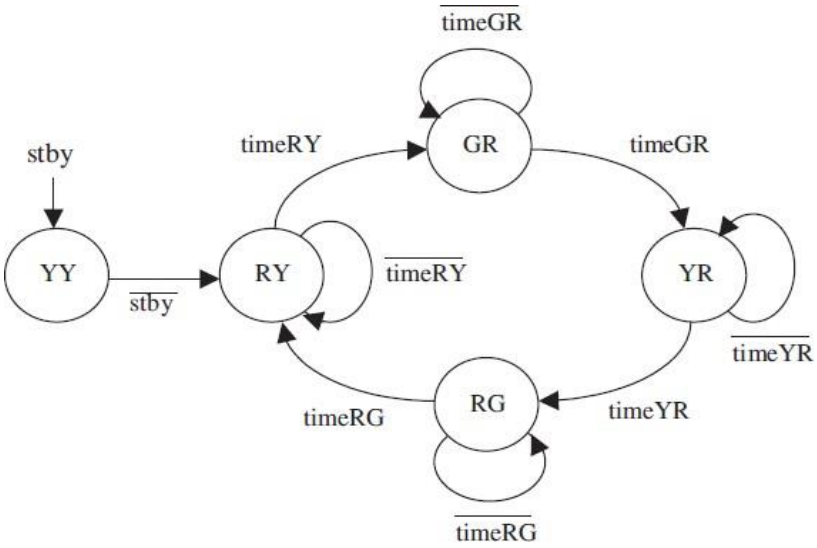


Figure 9.1 Finite State Machine Diagram of TLC

1. Generate the required structural VHDL code, include it in your project, and compile the circuit.
2. Use functional simulation to verify that your code is correct.

Practice Tasks: Transfer VHDL code into the FPGA board

1. Modify your design in part-1 for four way traffic light controller as

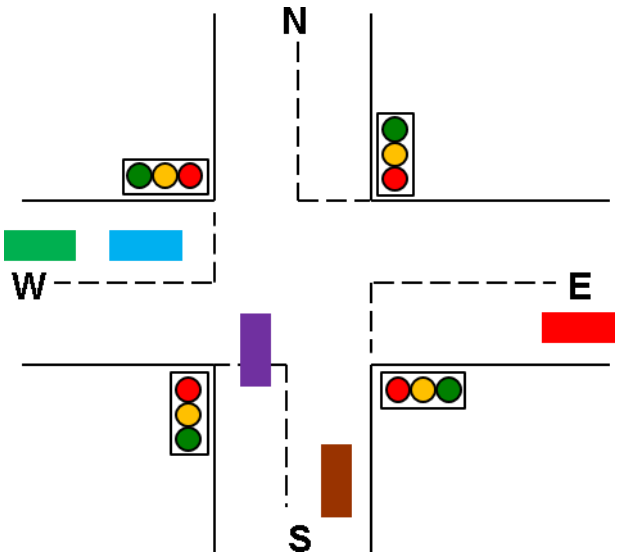


Figure 9.2 Four Way Traffic Light System

Modify your FSM and redraw it here

a) Generate the required structural VHDL code, include it in your project, and compile the circuit.
b) Use functional simulation to verify that your code is correct.

2. Add crosswalk requests to your state machine. By default, the "don't walk" signal should be on. When a crosswalk button is pressed, a walk signal should turn on the next time the parallel traffic signal turns green. (Hint: you can add another register to remember if the request buttons have been pressed during the current traffic cycle.) Present the updated state diagram and modify your VHDL design accordingly.

Rubric for lab assessment

The student performance for the assigned task during the lab session was:			
Excellent	The student completed all the tasks and showed the results without any help of the instructor.	4	
Good	The student completed all the tasks and showed the results with minimal help of the instructor.	3	
Average	The student partially completed the task and showed results.	2	
Worst	The student did not complete the task.	1	

Instructor Signature:_____ Date: _____